CS6190: Programming Assignment Report

Problem 7: Zero-Knowledge Proof of Correct Linear Model Inference with Binary Inputs

GADDAM HRUSHIKESH RAJ SAI CS25MTECH11026

SURESH KUMAR BADDIPUDI CS25MTECH13006

April 19, 2026

Abstract

This report details the architectural design, cryptographic foundations, and security evaluation of a C++ implementation for a multi-round Zero-Knowledge Proof (ZKP) protocol. The system enables a Prover to cryptographically convince a Verifier that the inference of a public linear machine learning model, defined as $y \equiv \sum w_i x_i + b \pmod{q}$, was computed correctly. The protocol enforces the constraint that all private inputs are strictly binary ($x_i \in \{0,1\}$) without leaking information about the inputs themselves. The implementation translates theoretical constructs—specifically Pedersen Commitments, Schnorr OR-proofs, and the Fiat-Shamir heuristic—into a standalone software architecture without reliance on external cryptographic libraries.

1 Introduction and Objectives

Privacy-Preserving Machine Learning (PPML) addresses the need to verify model inferences without revealing private input data to the evaluating server. A common challenge in this domain is proving that a client correctly applied a public model to their proprietary data.

The objective of this project was to implement a cryptographic protocol from scratch that guarantees the computational integrity of a linear inference. The system satisfies three strict constraints:

1. Prove the linear relation $y \equiv \sum_{i=1}^{n} w_i x_i + b \pmod{q}$.
2. Prove the binary domain of the inputs ($x_i \in \{0,1\}$).
3. Rely entirely on native algorithms, expressly forbidding the use of external cryptographic libraries such as OpenSSL or GMP.

2 Mathematical Preliminaries

2.1 The Algebraic Group and DLP Assumption

The protocol is instantiated over a cyclic subgroup $\mathbb{G}$ of prime order q, which is a subgroup of the multiplicative group of integers modulo a prime p ($\mathbb{Z}_p^*$). By Lagrange's Theorem, this requires q to divide $p-1$, establishing the Sylow subgroup equation: $p = nq + 1$.

The security of the protocol is predicated on the Discrete Logarithm Problem (DLP): given generators g and h in $\mathbb{G}$, it is computationally infeasible to find an integer k such that $g^k \equiv h \pmod{p}$.

2.2 Pedersen Commitments

To bind the Prover to their inputs without revealing them, the system utilizes Pedersen Commitments. For a private input x_i and a randomly sampled blinding factor $r_i \in \mathbb{Z}_q$, the commitment is computed as:

$$C_i \equiv g^{x_i} h^{r_i} \pmod{p} \tag{1}$$

Pedersen commitments are *perfectly hiding* (the probability distribution of C_i is uniform, revealing no information about x_i) and *computationally binding* (altering x_i after commitment requires solving the DLP). Furthermore, they possess an additive homomorphism: $C_1 \cdot C_2 \equiv g^{x_1+x_2} h^{r_1+r_2} \pmod{p}$.

3 Protocol Instantiation and Code Mapping

The system implements a Sigma protocol executed either interactively over TCP sockets or non-interactively via cryptographic hashing. The core logic is decoupled into `prover.cpp` and `verifier.cpp`, sharing mathematical and socket programming primitives from `zkp_core.hpp`.

3.1 The Binary Constraint (Schnorr OR-Proof)

To prove that $x_i \in \{0, 1\}$, the Prover must demonstrate knowledge of the randomness r_i for either the commitment C_i or C_i/g. This is achieved via a non-interactive Schnorr OR-proof utilizing witness simulation, executed within the `interactive_client` and `prove_fs` functions.

If the true input is $x_i = 0$, the Prover computes the true branch $A_0 \equiv h^{s_0} \pmod{p}$. The Prover simulates the false branch ($x_i = 1$) by selecting a challenge e_1 and response z_1 in advance. To compute the required commitment mathematically, the codebase invokes the custom modular inverse function:

$$A_1 \equiv h^{z_1} \cdot \texttt{egcd_inv}\left(\left(\frac{C_i}{g}\right)^{e_1}, p\right) \pmod{p} \tag{2}$$

Conversely, if the true input is $x_i = 1$, the Prover computes the true branch $A_1 \equiv h^{s_1} \pmod{p}$. The Prover simulates the false branch ($x_i = 0$) by selecting e_0 and z_0 in advance, reverse-engineering the commitment as:

$$A_0 \equiv h^{z_0} \cdot \texttt{egcd_inv}\left((C_i)^{e_0}, p\right) \pmod{p} \tag{3}$$

Upon receiving or generating a global challenge e, the Prover computes the true challenge for the non-simulated branch ($e_0 \equiv e - e_1 \pmod{q}$ or $e_1 \equiv e - e_0 \pmod{q}$). Within `verifier.cpp`, the `verify_fs` or `interactive_server` function explicitly enforces the split by asserting that `mod_add(e0, e1, q) == e`. Because the equations balance, the Verifier is convinced the bit is binary without distinguishing which branch was simulated.

3.2 Homomorphic Model Aggregation

A naive implementation would require a separate zero-knowledge proof for every weight w_i. Instead, the protocol exploits the additive homomorphism of the Pedersen commitments. Implemented via the `compute_D` function in the Verifier, the public parameters are aggregated into

a unified group element D:

$$D \equiv \left(\prod_{i=1}^{n} C_i^{w_i} \right) \cdot g^{b-y} \pmod{p} \tag{4}$$

$$\equiv \left(\prod_{i=1}^{n} (g^{x_i} h^{r_i})^{w_i} \right) g^{b-y} \pmod{p} \tag{5}$$

$$\equiv \left(g^{\sum w_i x_i} h^{\sum w_i r_i} \right) g^{b-y} \pmod{p} \tag{6}$$

Substituting the linear model relation $y = \sum w_i x_i + b$:

$$D \equiv \left(g^{y-b} h^{\sum w_i r_i} \right) g^{b-y} \pmod{p} \tag{7}$$

$$\equiv g^0 h^{\sum w_i r_i} \pmod{p} \equiv h^{\sum w_i r_i} \pmod{p} \tag{8}$$

By executing this aggregation, the g generators cancel out. The `compute_D` function reduces the multi-variable linear model inference to a single Schnorr Proof of Knowledge regarding the aggregated randomness $\rho = \sum w_i r_i \pmod{q}$, minimizing network payload and verification overhead.

4 Software Architecture and Engineering Constraints

4.1 Arbitrary-Precision Modular Arithmetic (`zkp_core.hpp`)

To prevent integer overflow during cryptographic operations, the `zkp_core.hpp` engine wraps all field arithmetic (e.g., `mod_mul`, `mod_pow`). Intermediate calculations cast native 64-bit unsigned integers (`uint64_t`) to unsigned 128-bit integers (`__int128`). Consequently, the finite field modulus p is strictly capped at $2^{64} - 1$, mathematically bounding the subgroup order q to a maximum of 63 bits.

The Extended Euclidean Algorithm implementation (`egcd_inv`) utilizes a *signed* 128-bit state to safely handle negative Bezout coefficients, preventing integer underflow during modular inverse calculations.

4.2 Algorithmic Parameter Generation (`generate_params.py`)

The Python setup script implements a standalone `is_prime` function utilizing the Miller-Rabin probabilistic primality test ($k = 40$ rounds). The `generate_subgroup_params` function enforces strict bit-shifting logic to guarantee that the generated prime q matches the exact byte boundaries requested via the CLI, ensuring accurate empirical boundary testing.

4.3 Non-Interactive Transformation (Fiat-Shamir)

For non-interactive proofs, the Fiat-Shamir heuristic utilizes a custom-built SHA-256 implementation. To translate the theoretical single-round protocol into a secure multi-round software architecture, hash inputs are strictly domain-separated.

First, the inputs are prefixed with contextual tags (e.g., `"bit|"` and `"lin|"`) to prevent cross-protocol collision attacks, ensuring an adversary cannot reuse a hash output generated in the OR-proof to satisfy the linear model proof. Second, the serialized state explicitly incorporates the current protocol round index and, for the OR-proofs, the specific bit index i. This guarantees cryptographic uniqueness across the transcript, strictly preventing an adversary from replaying commitments across different rounds or applying a single bit's challenge to multiple inputs.

5 Implementation Security Analysis

- **Completeness:** If the Prover is honest, the `interactive_client` or `prove_fs` successfully computes the simulated branch inverse via `egcd_inv`. The Verifier's `compute_D` aggregation perfectly cancels out the g generators, passing the final `lhs == rhs` assertions and yielding an ACCEPT state.
- **Soundness:** To prove soundness is enforced cryptographically rather than programmatically, the `load_witness` function intentionally omits client-side input validation (e.g., `if (bit != 0 && bit != 1)`). If an adversary inputs $x_i = 3$, the underlying commitment utilizes g^3. The algebraic relations in `interactive_server` or `verify_fs` natively fail to balance against a g^3 commitment for both the 0-branch and 1-branch simultaneously, safely rejecting the proof.
- **Special Honest-Verifier Zero-Knowledge (SHVZK):** The Verifier learns nothing about x_i. Because the false branch is simulated using the `rand_mod` RNG wrapper, the generated transcript distributions for $x = 0$ and $x = 1$ are statistically indistinguishable.

6 Empirical Testing and Verification

To empirically test the mathematical soundness and engineering stability of the C++ implementation, a five-stage test matrix was executed:

Test 1: Hardware Ceiling ($q = 63$ bits): Verified that the `__int128` arithmetic wrappers successfully handle maximum-capacity field elements without triggering silent integer overflow.

Test 2: Cryptographic Floor ($q = 6$ bits): Ensured that cyclic group generator derivation holds mathematically in extremely constrained finite fields.

Test 3: Standard Baseline ($q = 16$ bits): Demonstrated honest protocol execution representing typical application parameters.

Test 4: Soundness Attack (Tampered Witness): The Prover was provided an adversarial witness file containing a non-binary input ($x_i = 3$). The Prover lacked the cryptographic trapdoor required to simulate branches for g^3, resulting in an algebraic contradiction caught natively by the Verifier.

Test 5: Transcript Tampering (Fiat-Shamir Bypass): An intercepted non-interactive transcript (`fs_proof.txt`) was altered in transit. The `verify_fs` function autonomously reconstructed the domain-separated SHA-256 hashes, detected the transcript desynchronization, and rejected the proof.

7 Conclusion

The implementation provides a secure, zero-knowledge verification system for linear machine learning inferences. By homomorphically aggregating the model weights into a single verification element (D), the codebase minimizes verification overhead. By enforcing strict hardware constraints via `u128` math, simulating non-interactive transcripts via domain-separated hashing, and correctly invalidating adversarial inputs without relying on client-side checks, the system provides a robust, standalone instantiation of theoretical ZKP protocols.

References

[1] Fiat, A., & Shamir, A. (1986). How to Prove Yourself: Practical Solutions to Identification and Signature Problems. *Advances in Cryptology — CRYPTO '86*.

[2] Pedersen, T. P. (1991). Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing. *Advances in Cryptology — CRYPTO '91*.

[3] Cramer, R., Damgård, I., & Schoenmakers, B. (1994). Proofs of Partial Knowledge and Simplified Design of Witness Hiding Protocols. *Advances in Cryptology — CRYPTO '94*.

[4] Knuth, D. E. (1997). *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*.